

E19 — Data Distribution Generalization

Overview

This experiment investigates whether **Muon**'s spectral normalization remains effective across different **measurement matrix distributions**. In standard matrix sensing theory, Gaussian (normal) measurement matrices are typically assumed. However, in practice, the measurement process may follow different distributions. We test whether Muon's advantage over SGD generalizes to non-Gaussian distributions.

The distributions tested are:

1. **Normal**: Standard Gaussian (theoretical baseline)
2. **Uniform**: Uniformly distributed entries
3. **Rademacher**: Binary $\{-1, +1\}$ entries
4. **Sparse**: Sparse Gaussian (mostly zeros with few non-zero entries)
5. **Sphere**: Random vectors uniformly distributed on the unit sphere

Why this matters: Real-world measurement processes rarely follow perfect Gaussian distributions. If Muon's advantage depends critically on Gaussianity (e.g., for RIP guarantees), it may fail in practical settings. Testing across distributions assesses the **robustness and generalizability** of the spectral normalization mechanism.

Experiment ID: E19 | **Total runs:** 80 (2 algorithms *imes* 5 distributions *imes* 8 seeds)

Scientific Question

Hypothesis

- **Null Hypothesis (H_0):** Muon's relative performance vs SGD is identical across all measurement distributions — spectral normalization provides no differential advantage for any particular distribution.
- **Alternative Hypothesis (H_1):** Muon's spectral normalization interacts differently with different measurement distributions, leading to varying performance advantages.

Specific Questions

1. Does Muon maintain its convergence advantage across **all** tested distributions?
2. Are there distributions where SGD performs comparably or better than Muon?
3. Does the convergence rate (proportion of successful runs) vary by distribution?

4. Is computation time affected by distribution type?

Key Metrics

- K_ϵ : Iterations to reach threshold $\epsilon = 0.01$
- min loss: Best loss achieved
- I_{conv} : Convergence success flag
- time_g : Wall-clock time

Experimental Design

Parameter	Value
Problem	Matrix Sensing (MS)
Matrix dimension d	50
Target rank r	5
Learning rate η	0.01
Measurement samples m	$2dr = 500$
Measurement distributions	{normal, uniform, rademacher, sparse, sphere}
Noise	0 (noiseless)
Iteration budget	2000
Convergence threshold ϵ	0.01
Seeds per configuration	8
Algorithms	Muon-Exact, SGD

Distribution Definitions

Distribution	Description	Properties
normal	$[A_i]_{jk} \sim \mathcal{N}(0, 1)$	Theoretical standard; satisfies RIP
uniform	$[A_i]_{jk} \sim \text{Unif}(-1, 1)$	Bounded support; sub-Gaussian
rademacher	$[A_i]_{jk} \in \{-1, +1\}$ with equal probability	Discrete; sub-Gaussian
sparse	Mostly zeros; non-zeros $\sim \mathcal{N}(0, 1)$	Computationally efficient
sphere	Vectorized A_i uniformly on unit sphere	Isotropic but not independent entries

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# Color scheme
MUON_COLOR = '#2E86AB' # Blue
SGD_COLOR = '#F18F01' # Orange

# Set plotting style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11

print("Libraries imported successfully.")
```

```
In [ ]: # Load E19 data
df = pd.read_csv('../results_v3/E19_detailed_results.csv')

print(f"Shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
print(f"\nAlgorithms: {df['algo'].unique()}")
print(f"Distributions: {df['dist'].unique()}")
print(f"Seeds: {sorted(df['seed'].unique())}")
print("\nFirst few rows:")
df.head()
```

```
In [ ]: # Data quality check
print("=== Missing values ===")
print(df.isnull().sum())
print(f"\n=== K_epsilon summary by algorithm and distribution ===")
summary = df.groupby(['algo', 'dist'])['K_epsilon'].agg(['count', 'mean', 'std'])
print(summary)
print(f"\n=== Convergence status ===")
print(df.groupby(['algo', 'dist'])['I_conv'].mean())
```

Exploratory Data Analysis

We examine how convergence performance varies across measurement distributions for both algorithms, looking for any distribution where the relative Muon-SGD advantage breaks down.

```
In [ ]: # Detailed summary table
print("=" * 80)
print(f"{'Algorithm':<14} {'Dist':>12} {'N':>4} {'Mean K_eps':>12} {'Std K_e")
print("=" * 80)

dists = df['dist'].unique()
for algo in ['Muon-Exact', 'SGD']:
```

```

for dist in dists:
    sub = df[(df['algo'] == algo) & (df['dist'] == dist)]
    print(f"{algo:<14} {dist:>12} {len(sub):>4} {sub['K_epsilon'].mean():>12.1f} {sub['I_conv'].mean():>10.1f}")

# Best and worst distributions per algorithm
print("\n=== Best/Worst distribution per algorithm (by mean K_epsilon) ===")
for algo in ['Muon-Exact', 'SGD']:
    sub = df[df['algo'] == algo]
    mean_by_dist = sub.groupby('dist')['K_epsilon'].mean()
    print(f"{algo}: Best={mean_by_dist.idxmin()} (K_eps={mean_by_dist.min():>12.1f}) Worst={mean_by_dist.idxmax()} (K_eps={mean_by_dist.max():>12.1f})")

```

Comparative Analysis: Muon vs SGD

We compare the two algorithms head-to-head for each distribution, computing paired t-tests and efficiency ratios.

```

In [ ]: # Head-to-head comparison by distribution
print("=" * 95)
print(f"{dist:>12} {'Muon K_eps':>12} {'SGD K_eps':>12} {'Ratio':>8} {'t-stat':>12} {'p_val':>12} {'sig':>12}")
print("=" * 95)

for dist in dists:
    muon_k = df[(df['algo'] == 'Muon-Exact') & (df['dist'] == dist)]['K_epsilon']
    sgd_k = df[(df['algo'] == 'SGD') & (df['dist'] == dist)]['K_epsilon'].values

    ratio = muon_k.mean() / sgd_k.mean()
    t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
    sig = "" if p_val < 0.001 else "*" if p_val < 0.01 else "" if p_val < 0.05 else ""

    print(f"{dist:>12} {muon_k.mean():>12.1f} {sgd_k.mean():>12.1f} {ratio:>8.2f} {t_stat:>12.1f} {p_val:>12.1f} {sig}")

# ANOVA: Does distribution affect K_epsilon within each algorithm?
print("\n=== One-way ANOVA: Effect of distribution on K_epsilon ===")
for algo in ['Muon-Exact', 'SGD']:
    sub = df[df['algo'] == algo]
    groups = [sub[sub['dist'] == d]['K_epsilon'].values for d in sub['dist']]
    f_stat, p_val = stats.f_oneway(*groups)
    print(f"{algo}: F={f_stat:.3f}, p={p_val:.6f} ({'significant' if p_val < 0.05 else 'not significant'})")

```

Visualization 1: K_ϵ by Distribution (Grouped Bar)

This plot compares mean convergence iterations across measurement distributions for both algorithms.

```

In [ ]: fig, ax = plt.subplots(figsize=(12, 6))

x = np.arange(len(dists))
width = 0.35

```

```

muon_means = [df[(df['algo'] == 'Muon-Exact') & (df['dist'] == d)]['K_epsilon']
muon_stds = [df[(df['algo'] == 'Muon-Exact') & (df['dist'] == d)]['K_epsilon']
sgd_means = [df[(df['algo'] == 'SGD') & (df['dist'] == d)]['K_epsilon'].mean()
sgd_stds = [df[(df['algo'] == 'SGD') & (df['dist'] == d)]['K_epsilon'].std()

bars1 = ax.bar(x - width/2, muon_means, width, yerr=muon_stds, capsize=5,
               label='Muon-Exact', color=MUON_COLOR, edgecolor='black', alpha=0.85,
               label='SGD', color=SGD_COLOR, edgecolor='black', alpha=0.85,

ax.set_xlabel('Measurement Distribution', fontsize=13)
ax.set_ylabel(r'$K\_epsilon$ (Iterations to Convergence)', fontsize=13)
ax.set_title('Convergence Speed by Measurement Distribution (E19)', fontsize=13)
ax.set_xticks(x)
ax.set_xticklabels([d.capitalize() for d in dists])
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3, axis='y')

for i, (m, s) in enumerate(zip(muon_means, muon_stds)):
    ax.text(x[i] - width/2, m + s + 5, f'{m:.0f}', ha='center', va='bottom',
for i, (m, s) in enumerate(zip(sgd_means, sgd_stds)):
    ax.text(x[i] + width/2, m + s + 5, f'{m:.0f}', ha='center', va='bottom',

plt.tight_layout()
plt.savefig('E19_K_epsilon_by_dist.png', dpi=150, bbox_inches='tight')
plt.show()

```

Visualization 2: Convergence Rate by Distribution

This plot shows the proportion of runs that successfully converged within the iteration budget for each distribution.

```

In [ ]: fig, ax = plt.subplots(figsize=(12, 6))

conv_data = df.groupby(['algo', 'dist'])['I_conv'].mean().reset_index()
muon_conv = conv_data[conv_data['algo'] == 'Muon-Exact']
sgd_conv = conv_data[conv_data['algo'] == 'SGD']

x = np.arange(len(dists))
width = 0.35

bars1 = ax.bar(x - width/2, muon_conv['I_conv'], width, label='Muon-Exact',
bars2 = ax.bar(x + width/2, sgd_conv['I_conv'], width, label='SGD', color=SGD_COLOR,

ax.set_xlabel('Measurement Distribution', fontsize=13)
ax.set_ylabel('Convergence Rate', fontsize=13)
ax.set_title('Convergence Success Rate by Distribution (E19)', fontsize=14,
ax.set_xticks(x)
ax.set_xticklabels([d.capitalize() for d in dists])
ax.set_ylim(0, 1.05)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3, axis='y')

for bar in bars1:

```

```

        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.01, f'{height:.2f}'
                ha='center', va='bottom', fontsize=9, fontweight='bold')
    for bar in bars2:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.01, f'{height:.2f}'
                ha='center', va='bottom', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('E19_convergence_rate_by_dist.png', dpi=150, bbox_inches='tight')
plt.show()

```

Visualization 3: Wall-Clock Time by Distribution

This plot compares the actual computation time across distributions, which may reveal practical considerations beyond iteration count.

```

In [ ]: fig, ax = plt.subplots(figsize=(12, 6))

muon_times = [df[(df['algo'] == 'Muon-Exact') & (df['dist'] == d)]['time_s']]
muon_time_stds = [df[(df['algo'] == 'Muon-Exact') & (df['dist'] == d)]['time_s'].std()]
sgd_times = [df[(df['algo'] == 'SGD') & (df['dist'] == d)]['time_s'].mean()]
sgd_time_stds = [df[(df['algo'] == 'SGD') & (df['dist'] == d)]['time_s'].std()]

bars1 = ax.bar(x - width/2, muon_times, width, yerr=muon_time_stds, capsize=5,
               label='Muon-Exact', color=MUON_COLOR, edgecolor='black', alpha=0.85)
bars2 = ax.bar(x + width/2, sgd_times, width, yerr=sgd_time_stds, capsize=5,
               label='SGD', color=SGD_COLOR, edgecolor='black', alpha=0.85)

ax.set_xlabel('Measurement Distribution', fontsize=13)
ax.set_ylabel('Wall-Clock Time (seconds)', fontsize=13)
ax.set_title('Computation Time by Distribution (E19)', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels([d.capitalize() for d in dists])
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('E19_time_by_dist.png', dpi=150, bbox_inches='tight')
plt.show()

```

Statistical Tests

Paired t-tests at each distribution, plus ANOVA testing whether the distribution has a significant effect on convergence speed within each algorithm.

```

In [ ]: def cohens_d(x, y, paired=True):
        if paired:
            diff = x - y
            return diff.mean() / diff.std(ddof=1) if diff.std(ddof=1) > 0 else 0
        else:

```

```

        pooled_std = np.sqrt(((len(x)-1)*x.var(ddof=1) + (len(y)-1)*y.var(ddof=1)) / (len(x)+len(y)-2))
        return (x.mean() - y.mean()) / pooled_std if pooled_std > 0 else 0

# Paired t-tests: Muon vs SGD per distribution
print("=" * 95)
print(f"{'Dist':>12} {'Muon mean':>11} {'SGD mean':>11} {'Diff':>9} {'Cohen d':>9} {'t_stat':>9} {'p_val':>9} {'sig':>5}")
print("=" * 95)

for dist in dists:
    muon_k = df[(df['algo'] == 'Muon-Exact') & (df['dist'] == dist)][['K_epsilon', 't_stat', 'p_val', 'sig']]
    sgd_k = df[(df['algo'] == 'SGD') & (df['dist'] == dist)][['K_epsilon', 't_stat', 'p_val', 'sig']]
    d = cohens_d(muon_k, sgd_k, paired=True)
    t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
    sig = "" if p_val < 0.001 else "*" if p_val < 0.01 else "" if p_val < 0.05 else ""
    print(f"{'Dist':>12} {'Muon_k.mean():>11.1f} {'sgd_k.mean():>11.1f} {'muon_k.mean()-sgd_k.mean():>9.3f} {'t_stat:>9.3f} {'p_val:>13.6f} {'sig:>5}")

# Post-hoc pairwise comparisons of distributions within each algorithm
print("\n=== Pairwise distribution comparisons (within Muon) ===")
from itertools import combinations
muon_df = df[df['algo'] == 'Muon-Exact']
for d1, d2 in combinations(dists, 2):
    g1 = muon_df[muon_df['dist'] == d1]['K_epsilon'].values
    g2 = muon_df[muon_df['dist'] == d2]['K_epsilon'].values
    t_stat, p_val = stats.ttest_ind(g1, g2)
    sig = "" if p_val < 0.001 else "*" if p_val < 0.01 else "" if p_val < 0.05 else ""
    print(f"    {d1} vs {d2}: t={t_stat:+.3f}, p={p_val:.4f} {sig}")

```

Conclusions & Interpretation

Summary of Findings

1. **Distribution Robustness:** If Muon maintains a significant advantage (lower K_ϵ) across all tested distributions, this demonstrates that spectral normalization is a robust optimization mechanism that does not critically depend on Gaussian measurement matrices.
2. **Best/Worst Distributions:** The summary reveals which distributions are easiest and hardest for each algorithm. If both algorithms struggle with the same distributions (e.g., sparse or rademacher), this indicates an inherent problem difficulty rather than an algorithm-specific limitation.
3. **Convergence Reliability:** The convergence rate plot reveals whether any distribution causes systematic failures for either algorithm. This is critical for practical deployment.
4. **Computation Time:** If time varies significantly by distribution, this may be due to different convergence speeds (reflected in K_ϵ) or per-iteration cost differences.

Key Takeaway

This experiment tests the **generalizability** of Muon's spectral normalization. If the efficiency ratio ρ is consistently below 1 across all distributions, this is strong evidence that Muon's advantage is not an artifact of Gaussian assumptions but a genuine optimization improvement.

Limitations

- Only 5 distributions tested
- 8 seeds per configuration
- All distributions are centered and zero-mean; heavy-tailed or skewed distributions not tested
- The sparse distribution's sparsity level is fixed and not varied